

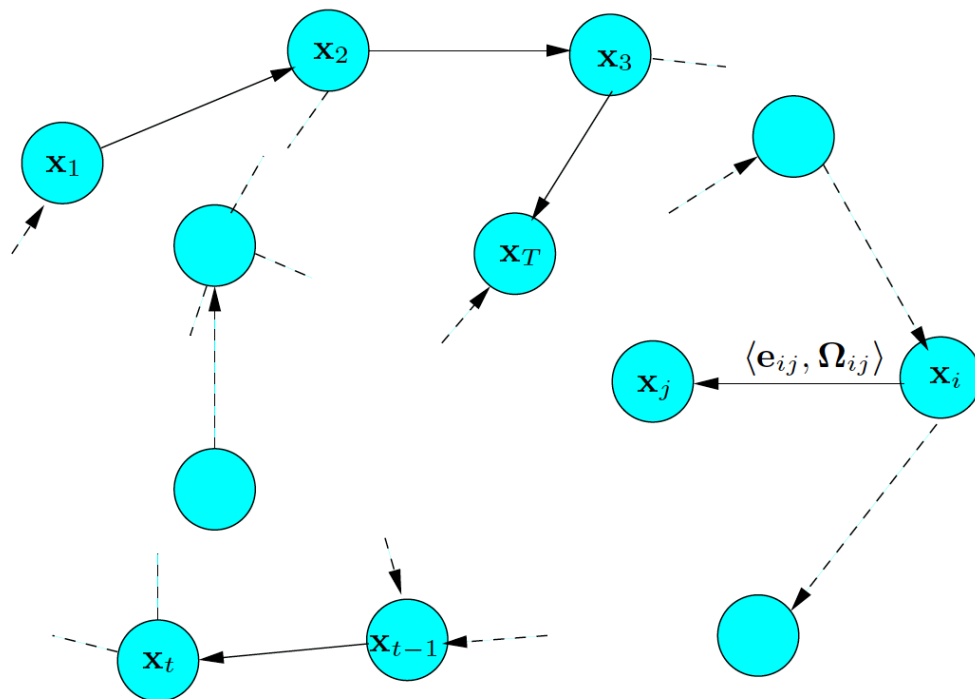


Graph SLAM

1. Graph SLAM Derivation
2. Graph SLAM Algorithm

二：图优化模型

A general graph model without differing the control and observation



$$\mathbf{e}_{ij}(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{z}_{ij} - \hat{\mathbf{z}}_{ij}(\mathbf{x}_i, \mathbf{x}_j)$$

$$\mathbf{F}(\mathbf{x}) = \sum_{\langle i, j \rangle \in \mathcal{C}} \underbrace{\mathbf{e}_{ij}^T \mathbf{\Omega}_{ij} \mathbf{e}_{ij}}_{\mathbf{F}_{ij}}, \quad \mathbf{x}^* = \underset{\mathbf{x}}{\operatorname{argmin}} \mathbf{F}(\mathbf{x})$$

图优化问题的求解

$$\mathbf{F}(\mathbf{x}) = \sum_{\langle i,j \rangle \in \mathcal{C}} \underbrace{\mathbf{e}_{ij}^T \boldsymbol{\Omega}_{ij} \mathbf{e}_{ij}}_{\mathbf{F}_{ij}},$$

最小化无约束非线性目标函数：

- 梯度下降
- ADMM
- 牛顿法等

用高斯牛顿法求解图优化问题

图优化的目标函数的特殊的特点，使得高斯牛顿法成为主要求解方法

$$\begin{aligned} \mathbf{e}_{ij}(\check{\mathbf{x}}_i + \Delta \mathbf{x}_i, \check{\mathbf{x}}_j + \Delta \mathbf{x}_j) &= \mathbf{e}_{ij}(\check{\mathbf{x}} + \Delta \mathbf{x}) \\ &\simeq \mathbf{e}_{ij} + \mathbf{J}_{ij} \Delta \mathbf{x}. \end{aligned}$$

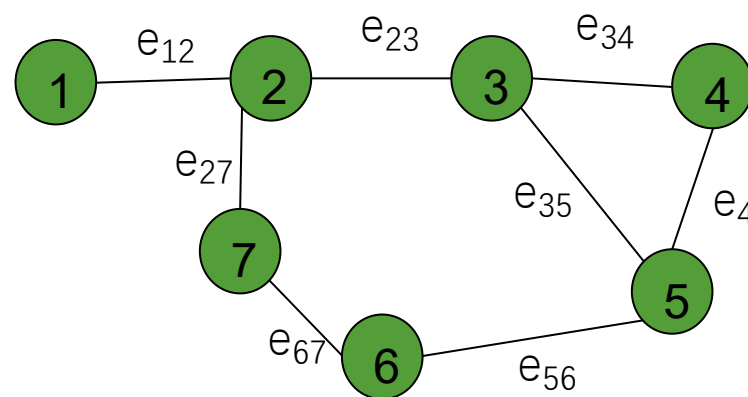
$$\mathbf{J}_{ij} = \begin{pmatrix} 0 \cdots 0 & \underbrace{\mathbf{A}_{ij}}_{\text{node } i} & 0 \cdots 0 & \underbrace{\mathbf{B}_{ij}}_{\text{node } j} & 0 \cdots 0 \end{pmatrix}$$

$$e_{23}(\mathbf{x}) = [x_2 - g(u_1, x_1)]$$

$$J_{23} = \frac{\partial e_{23}(\mathbf{x} + \Delta \mathbf{x})}{\partial \mathbf{x}}$$

$$= \frac{\partial [x_2 + \Delta x_2 - g(u_1, x_1 + \Delta x_1)]}{\partial \mathbf{x}}$$

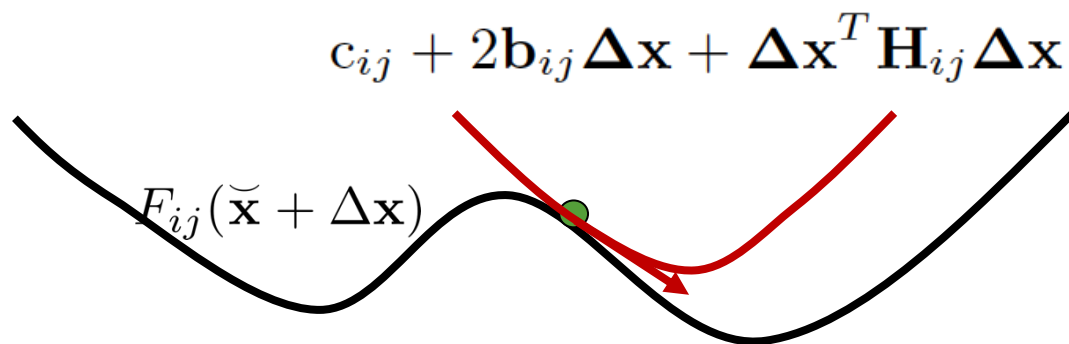
$$= \left(\frac{-\partial g(u_1, x_1 + \Delta x_1)}{\partial x_1}, \frac{\partial (x_2 + \Delta x_2)}{\partial x_2}, 0, \dots, 0 \right)$$



对目标函数进行二次近似

$$\begin{aligned} & \mathbf{F}_{ij}(\check{\mathbf{x}} + \Delta \mathbf{x}) \\ &= \mathbf{e}_{ij}(\check{\mathbf{x}} + \Delta \mathbf{x})^T \boldsymbol{\Omega}_{ij} \mathbf{e}_{ij}(\check{\mathbf{x}} + \Delta \mathbf{x}) \\ &\simeq (\mathbf{e}_{ij} + \mathbf{J}_{ij} \Delta \mathbf{x})^T \boldsymbol{\Omega}_{ij} (\mathbf{e}_{ij} + \mathbf{J}_{ij} \Delta \mathbf{x}) \\ &= \underbrace{\mathbf{e}_{ij}^T \boldsymbol{\Omega}_{ij} \mathbf{e}_{ij}}_{c_{ij}} + 2 \underbrace{\mathbf{e}_{ij}^T \boldsymbol{\Omega}_{ij} \mathbf{J}_{ij}}_{\mathbf{b}_{ij}} \Delta \mathbf{x} + \Delta \mathbf{x}^T \underbrace{\mathbf{J}_{ij}^T \boldsymbol{\Omega}_{ij} \mathbf{J}_{ij}}_{\mathbf{H}_{ij}} \Delta \mathbf{x} \\ &= c_{ij} + 2\mathbf{b}_{ij} \Delta \mathbf{x} + \Delta \mathbf{x}^T \mathbf{H}_{ij} \Delta \mathbf{x} \end{aligned}$$

在任意时刻，用一阶Taylor 展开近似目标函数，得到一个二次函数进行局部近似



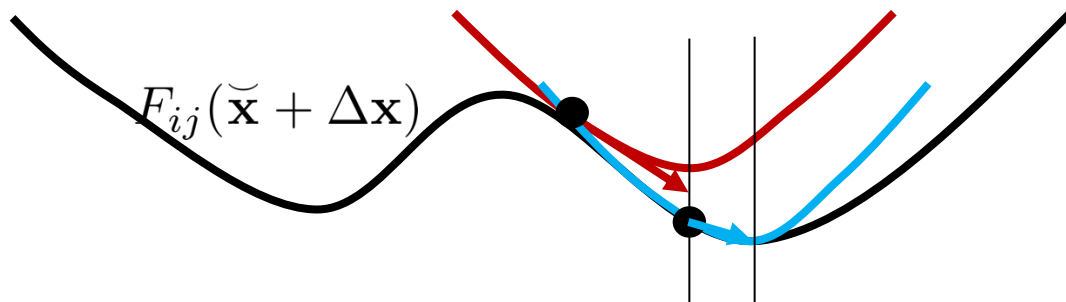
$$\begin{aligned}
\mathbf{F}(\breve{\mathbf{x}} + \Delta \mathbf{x}) &= \sum_{\langle i,j \rangle \in \mathcal{C}} \mathbf{F}_{ij}(\breve{\mathbf{x}} + \Delta \mathbf{x}) \\
&\simeq \sum_{\langle i,j \rangle \in \mathcal{C}} c_{ij} + 2\mathbf{b}_{ij} \Delta \mathbf{x} + \Delta \mathbf{x}^T \mathbf{H}_{ij} \Delta \mathbf{x} \\
&= c + 2\mathbf{b}^T \Delta \mathbf{x} + \Delta \mathbf{x}^T \mathbf{H} \Delta \mathbf{x}.
\end{aligned}$$



$$\mathbf{H} \Delta \mathbf{x}^* = -\mathbf{b}.$$



$$\mathbf{x}^* = \breve{\mathbf{x}} + \Delta \mathbf{x}^*$$



H矩阵和b矩阵的构造

$$\mathbf{H}_{ij} = \mathbf{J}_{ij}^T \boldsymbol{\Omega}_{ij} \mathbf{J}_{ij}$$

$$\mathbf{J}_{ij} = \begin{pmatrix} \mathbf{0} \cdots \mathbf{0} & \underbrace{\mathbf{A}_{ij}}_{\text{node } i} & \mathbf{0} \cdots \mathbf{0} & \underbrace{\mathbf{B}_{ij}}_{\text{node } j} & \mathbf{0} \cdots \mathbf{0} \end{pmatrix}$$

$$\mathbf{A}_{i,j} = \frac{\partial \mathbf{e}(\mathbf{x}_i, \mathbf{x}_j)}{\mathbf{x}_i} \quad \mathbf{B}_{i,j} = \frac{\partial \mathbf{e}(\mathbf{x}_i, \mathbf{x}_j)}{\mathbf{x}_j}$$

$$\mathbf{H}_{ij} = \begin{pmatrix} \ddots & & & & \\ & \mathbf{A}_{ij}^T \boldsymbol{\Omega}_{ij} \mathbf{A}_{ij} & \cdots & \mathbf{A}_{ij}^T \boldsymbol{\Omega}_{ij} \mathbf{B}_{ij} & \\ & \vdots & \ddots & \vdots & \\ & \mathbf{B}_{ij}^T \boldsymbol{\Omega}_{ij} \mathbf{A}_{ij} & \cdots & \mathbf{B}_{ij}^T \boldsymbol{\Omega}_{ij} \mathbf{B}_{ij} & \\ & & & \ddots & \end{pmatrix}$$

$$\mathbf{H} = \sum_{(i,j) \in E} \mathbf{H}_{ij}$$

$$\mathbf{b}_{ij} = \mathbf{e}_{ij}^T \boldsymbol{\Omega}_{ij} \mathbf{J}_{ij}$$

$$\mathbf{b}_{ij} = \begin{pmatrix} \vdots \\ \mathbf{A}_{ij}^T \boldsymbol{\Omega}_{ij} \mathbf{e}_{ij} \\ \vdots \\ \mathbf{B}_{ij}^T \boldsymbol{\Omega}_{ij} \mathbf{e}_{ij} \\ \vdots \end{pmatrix}$$

$$\mathbf{b} = \sum_{(i,j) \in E} \mathbf{b}_{ij}$$

1. 构造目标函数，等价于构造图中的边

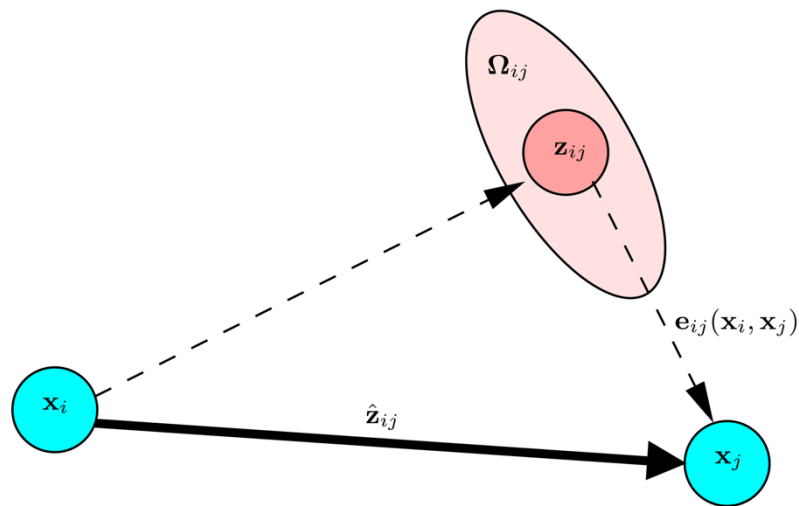
1. 图中有多种边：

1. 到Landmark的测量边

- 距离测量
- 角度测量
- 既有距离又有角度

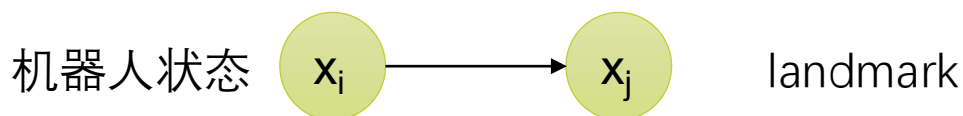
2. 机器人连续移动状态之间的

- 基于运动模型描述
- 没有运动模型时，可以基于位姿变换描述



到Landmark的测量边

1. 机器人到landmark的测量



2. 仅有距离测量时:

- 实际测量值: $d_{i,j}$
- 预测测量值: $\tilde{z}_{i,j} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$
- 此边的测量与基于当前状态预测的误差

$$e_{ij}(x_i, x_j) = d_{i,j} - \tilde{z}_{i,j} = d_{i,j} - \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

- 测量的方差:
- 根据测距的误差来给

$$\Omega_{i,j} = \frac{1}{\sigma_d^2}$$

到Landmark的测量边

• 3. 仅有角度测量时:

- 实际测量值: $\alpha_{i,j}$

- 预测测量值: $\tilde{z}_{i,j} = \arctan \frac{y_i - y_j}{x_i - x_j} - \theta_i$

- 此边的测量与基于当前状态预测的误差

$$e_{ij}(x_i, x_j) = \alpha_{i,j} - \tilde{z}_{i,j} = \alpha_{i,j} - \arctan \frac{y_i - y_j}{x_i - x_j} + \theta_i \quad \Omega_{i,j} = \frac{1}{\sigma_\alpha^2}$$

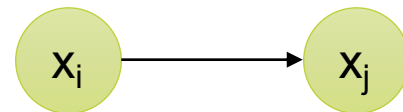
• 4. 既有距离又有角度时:

- 实际测量: $z_{i,j} = (d_{i,j}, \alpha_{i,j})^T$

- 预测测量值: $\tilde{z}_{i,j} = \begin{bmatrix} \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \\ \arctan \frac{y_i - y_j}{x_i - x_j} - \theta_i \end{bmatrix}$

- 此边的测量与基于当前状态预测的误差

$$e_{ij}(x_i, x_j) = \begin{bmatrix} d_{i,j} - \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \\ \alpha_{i,j} - \arctan \frac{y_i - y_j}{x_i - x_j} + \theta_i \end{bmatrix} \quad \Omega_{i,j} = \begin{bmatrix} \frac{1}{\sigma_d^2} & 0 \\ 0 & \frac{1}{\sigma_\alpha^2} \end{bmatrix}$$



机器人状态

landmark

```

while  $\neg$ converged do
   $\mathbf{b} \leftarrow \mathbf{0}$      $\mathbf{H} \leftarrow \mathbf{0}$ 
  for all  $\langle \mathbf{e}_{ij}, \mathbf{\Omega}_{ij} \rangle \in \mathcal{C}$  do
    // Compute the Jacobians  $\mathbf{A}_{ij}$  and  $\mathbf{B}_{ij}$  of the error
    function
     $\mathbf{A}_{ij} \leftarrow \left. \frac{\partial \mathbf{e}_{ij}(\mathbf{x})}{\partial \mathbf{x}_i} \right|_{\mathbf{x}=\check{\mathbf{x}}}$      $\mathbf{B}_{ij} \leftarrow \left. \frac{\partial \mathbf{e}_{ij}(\mathbf{x})}{\partial \mathbf{x}_j} \right|_{\mathbf{x}=\check{\mathbf{x}}}$ 
    // compute the contribution of this constraint to the
    linear system
     $\mathbf{H}_{[ii]} += \mathbf{A}_{ij}^T \mathbf{\Omega}_{ij} \mathbf{A}_{ij}$      $\mathbf{H}_{[ij]} += \mathbf{A}_{ij}^T \mathbf{\Omega}_{ij} \mathbf{B}_{ij}$ 
     $\mathbf{H}_{[ji]} += \mathbf{B}_{ij}^T \mathbf{\Omega}_{ij} \mathbf{A}_{ij}$      $\mathbf{H}_{[jj]} += \mathbf{B}_{ij}^T \mathbf{\Omega}_{ij} \mathbf{B}_{ij}$ 
    // compute the coefficient vector
     $\mathbf{b}_{[i]} += \mathbf{A}_{ij}^T \mathbf{\Omega}_{ij} \mathbf{e}_{ij}$      $\mathbf{b}_{[j]} += \mathbf{B}_{ij}^T \mathbf{\Omega}_{ij} \mathbf{e}_{ij}$ 
  end for
  // keep the first node fixed
   $\mathbf{H}_{[11]} += \mathbf{I}$ 
  // solve the linear system using sparse Cholesky factor-
  ization
   $\Delta \mathbf{x} \leftarrow \text{solve}(\mathbf{H} \Delta \mathbf{x} = -\mathbf{b})$ 
  // update the parameters
   $\check{\mathbf{x}} += \Delta \mathbf{x}$ 
end while

```

基于稀疏Cholesky分解求解

$$\mathbf{A}\mathbf{X} = \mathbf{b}$$

- Cholesky decomposes $\mathbf{A}$ into $\mathbf{A} = \mathbf{L}\mathbf{L}^T$

$$\begin{aligned} \begin{bmatrix} a_{11} & A_{21}^T \\ A_{21} & A_{22} \end{bmatrix} &= \begin{bmatrix} l_{11} & 0 \\ L_{21} & L_{22} \end{bmatrix} \begin{bmatrix} l_{11} & L_{21}^T \\ 0 & L_{22}^T \end{bmatrix} \\ &= \begin{bmatrix} l_{11}^2 & l_{11}L_{21}^T \\ l_{11}L_{21} & L_{21}L_{21}^T + L_{22}L_{22}^T \end{bmatrix} \end{aligned}$$

algorithm

- determine l_{11} and L_{21} :

$$l_{11} = \sqrt{a_{11}}, \quad L_{21} = \frac{1}{l_{11}}A_{21}$$

- compute L_{22} from

$$A_{22} - L_{21}L_{21}^T = L_{22}L_{22}^T$$

this is a Cholesky factorization of order $n - 1$

Example

$$\begin{bmatrix} 25 & 15 & -5 \\ 15 & 18 & 0 \\ -5 & 0 & 11 \end{bmatrix} = \begin{bmatrix} l_{11} & 0 & 0 \\ l_{21} & l_{22} & 0 \\ l_{31} & l_{32} & l_{33} \end{bmatrix} \begin{bmatrix} l_{11} & l_{21} & l_{31} \\ 0 & l_{22} & l_{32} \\ 0 & 0 & l_{33} \end{bmatrix}$$

- first column of L

$$\begin{bmatrix} 25 & 15 & -5 \\ 15 & 18 & 0 \\ -5 & 0 & 11 \end{bmatrix} = \begin{bmatrix} 5 & 0 & 0 \\ 3 & l_{22} & 0 \\ -1 & l_{32} & l_{33} \end{bmatrix} \begin{bmatrix} 5 & 3 & -1 \\ 0 & l_{22} & l_{32} \\ 0 & 0 & l_{33} \end{bmatrix}$$

- second column of L

$$\begin{bmatrix} 18 & 0 \\ 0 & 11 \end{bmatrix} - \begin{bmatrix} 3 \\ -1 \end{bmatrix} \begin{bmatrix} 3 & -1 \end{bmatrix} = \begin{bmatrix} l_{22} & 0 \\ l_{32} & l_{33} \end{bmatrix} \begin{bmatrix} l_{22} & l_{32} \\ 0 & l_{33} \end{bmatrix}$$

$$\begin{bmatrix} 9 & 3 \\ 3 & 10 \end{bmatrix} = \begin{bmatrix} 3 & 0 \\ 1 & l_{33} \end{bmatrix} \begin{bmatrix} 3 & 1 \\ 0 & l_{33} \end{bmatrix}$$

- third column of L : $10 - 1 = l_{33}^2$, i.e., $l_{33} = 3$

conclusion:

$$\begin{bmatrix} 25 & 15 & -5 \\ 15 & 18 & 0 \\ -5 & 0 & 11 \end{bmatrix} = \begin{bmatrix} 5 & 0 & 0 \\ 3 & 3 & 0 \\ -1 & 1 & 3 \end{bmatrix} \begin{bmatrix} 5 & 3 & -1 \\ 0 & 3 & 1 \\ 0 & 0 & 3 \end{bmatrix}$$

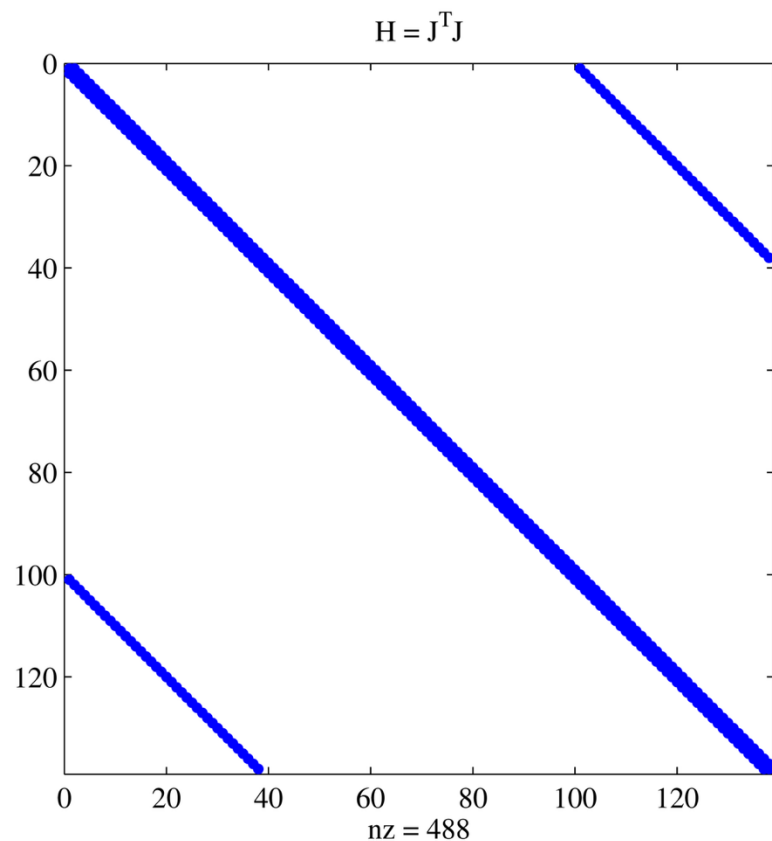
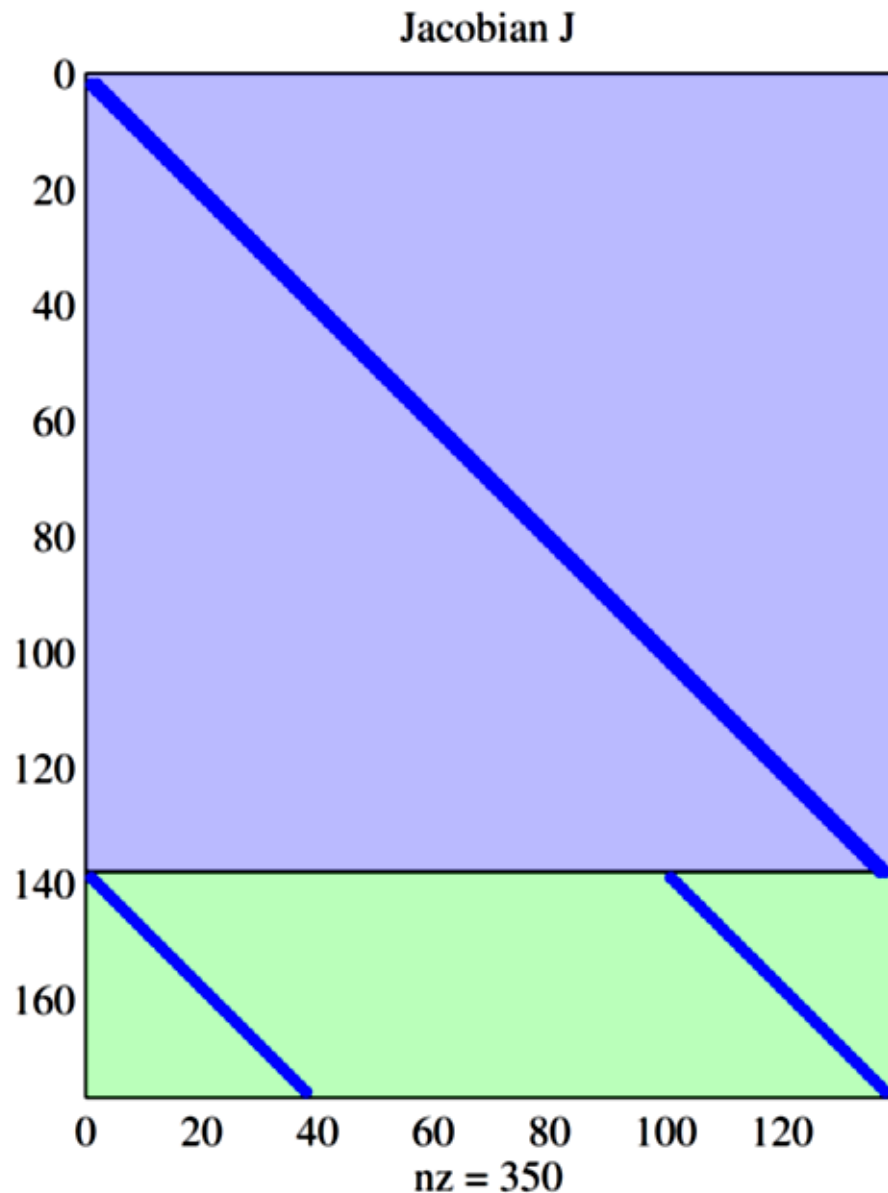
在Cholesky分解之后，原求解线性方程组问题变为：

$$\mathbf{LL}^T \Delta \mathbf{X} = \mathbf{b} \quad \Rightarrow \quad \begin{aligned} \mathbf{Ly} &= \mathbf{b} \\ \mathbf{L}^T \Delta \mathbf{X} &= \mathbf{y} \end{aligned}$$

变得非常容易求解：

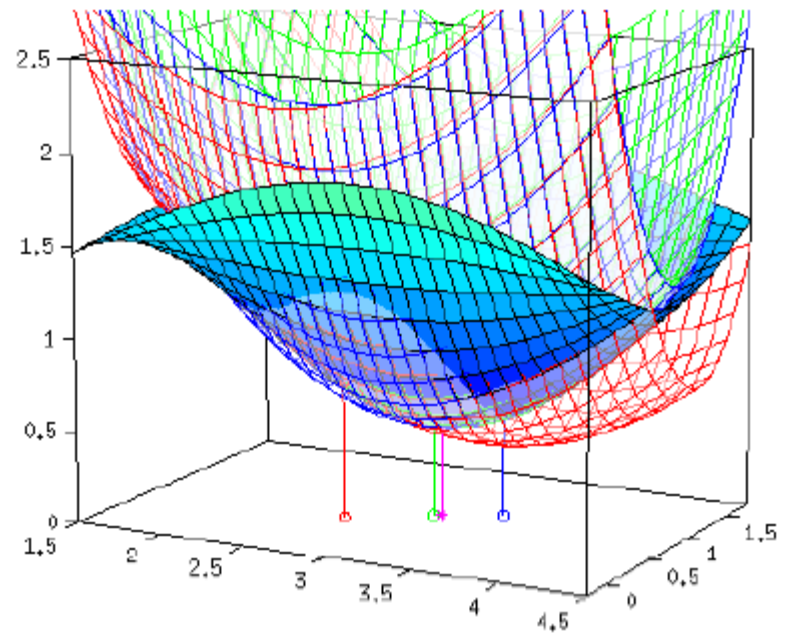
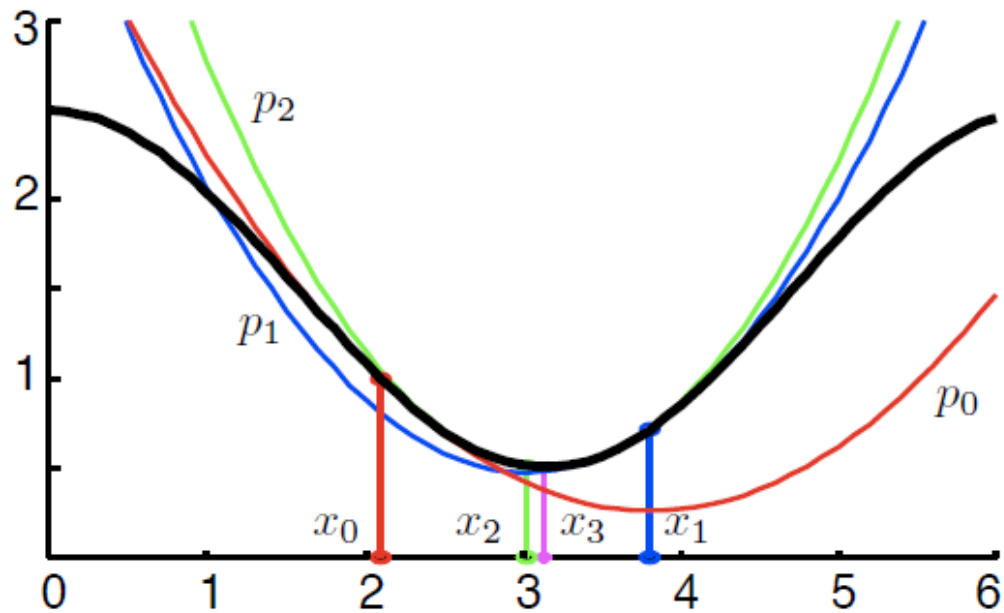
$$\begin{bmatrix} 5 & 0 & 0 \\ 3 & 3 & 0 \\ -1 & 1 & 3 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 0.9 \\ 0.5 \end{bmatrix} \quad \begin{bmatrix} 5 & 3 & -1 \\ 0 & 3 & 1 \\ 0 & 0 & 3 \end{bmatrix} \begin{bmatrix} \Delta x_1 \\ \Delta x_2 \\ \Delta x_3 \end{bmatrix} = \begin{bmatrix} 0.2 \\ 0.1 \\ 0.2 \end{bmatrix}$$

图优化的高效计算



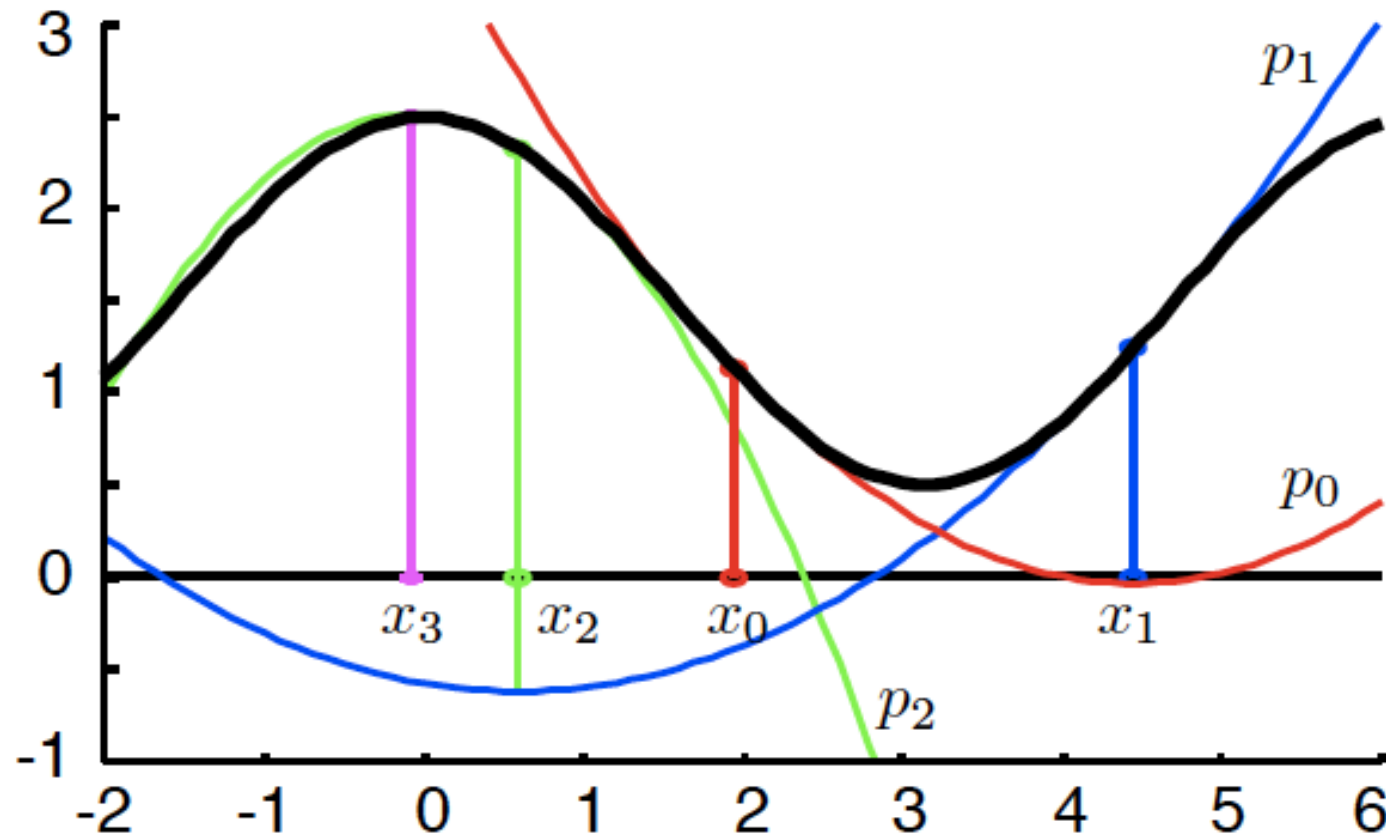
例如在G2O的数据示例中H: 10500 * 10500
但非零元素只占0.17%，基于Cholesky可以有效计算

Gauss-Newton Method

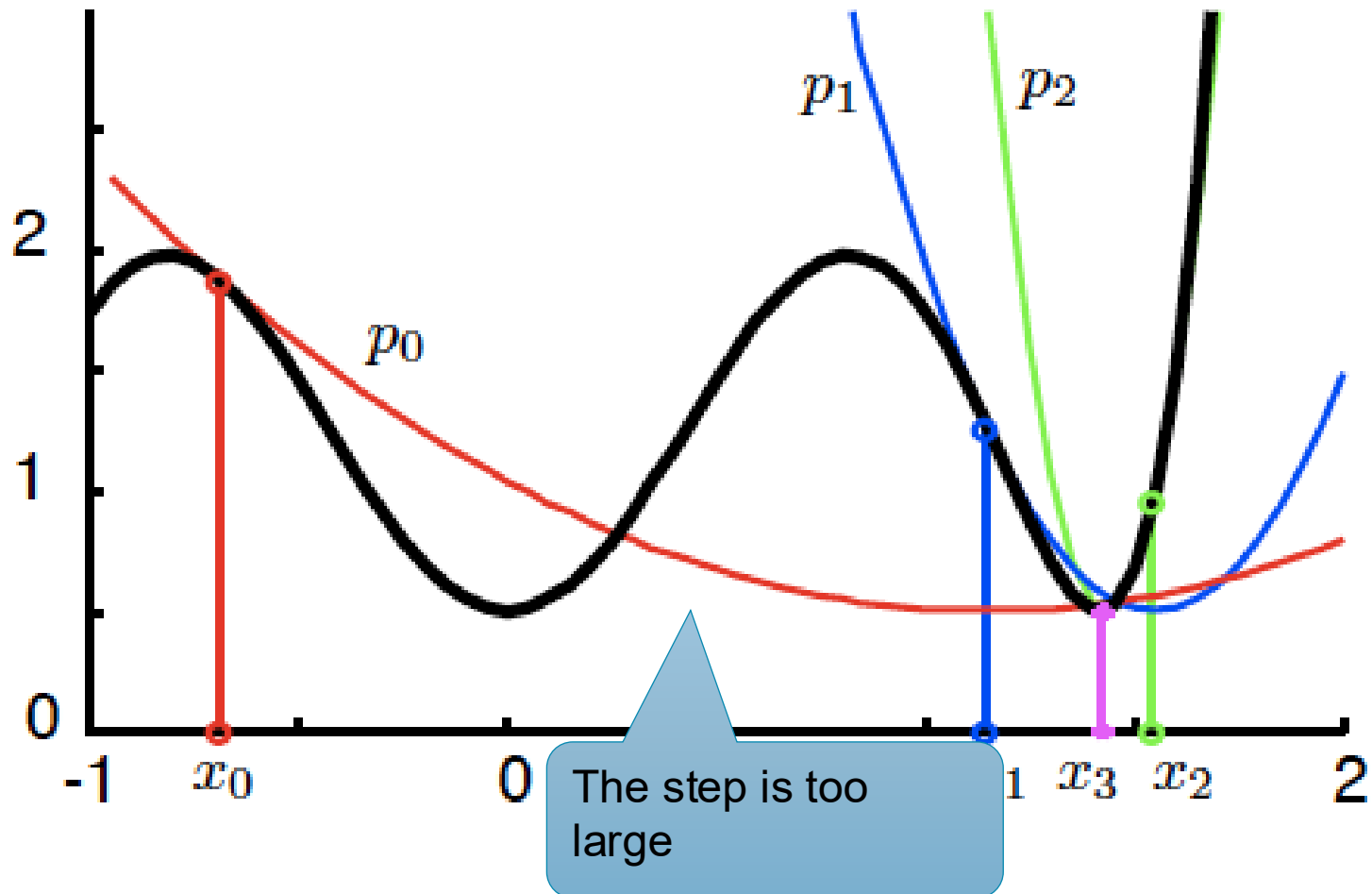


Two Potential Problems of Newton Method

1. The step length may be too big to escape from the minimum
2. In concave zones, the step departs towards the opposite direction



Large step; Falling into local minimum



Improved Gauss-Newton Algorithms

Levenberg:

$$\Delta \mathbf{x}_L^* = -\alpha (\mathbf{H} + \lambda \mathbf{I})^{-1} \nabla \mathbf{F}^\top$$

Marquardt:

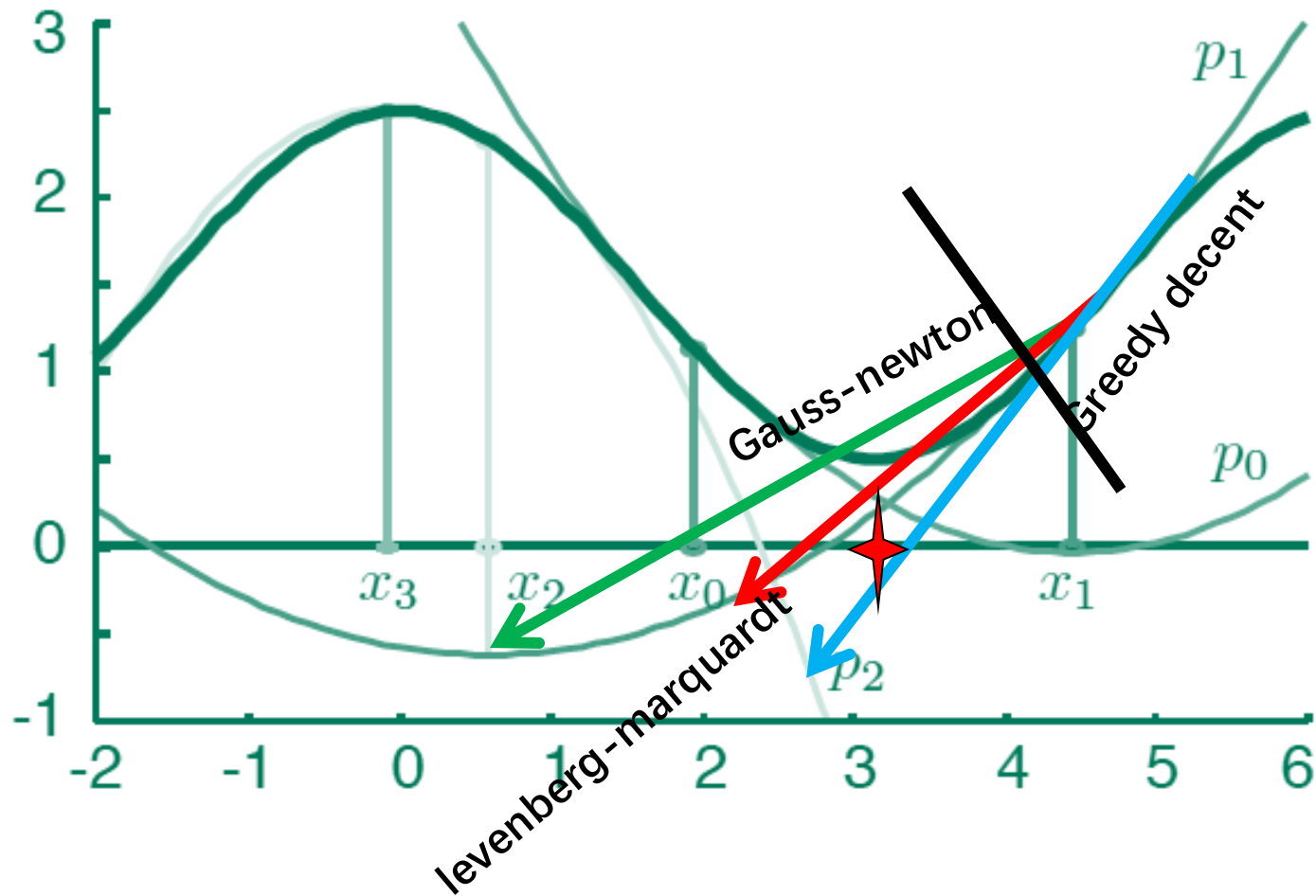
$$\Delta \mathbf{x}_{LM}^* = -\alpha (\mathbf{H} + \lambda \operatorname{diag}(\mathbf{H}))^{-1} \nabla \mathbf{F}^\top$$

Damped Newton Method:

$$\lambda^* = \arg \min_{\lambda} f \left(x_k - \lambda H^{-1} \Delta F^T \right)$$

$$\Delta \mathbf{x}_{DNM}^* = -\lambda^* H^{-1} \Delta F^T$$

Improved Gauss-Newton Algorithms



Graph-Slam 开源项目

- **Ceres:** [A Large Scale Non-linear Optimization Library](http://ceres-solver.org/)
- <http://ceres-solver.org/>
- **G2O:** General graph optimization
- <https://github.com/RainerKuemmerle/g2o>
- **GTSAM:** Georgia Tech Smoothing and Mapping Library
- <https://github.com/borglab/gtsam>
- **miniSAM:** A Flexible Factor Graph Non-linear Least Squares Optimization framework.
- <https://github.com/dongjing3309/minisam>

开源库名称	特点	语言	stars
Ceres	求解非线性优化的大型开源库	C++	1.7K
G2O	求解图优化问题的开源库	C++, matlab, python	1.7k
GTSAM	基于graph-slam定位和建图的开源库	C++, 现有python和matlab wrapper	725
miniSAM	类似于GTSAM的轻量级开源实现	python	224